

PROYECTO DE COMPILADORES



Alejandro Carrión Jordán (a.carrionjordan@um.es)
Borja Sancho Fernández (borja.s.f@um.es)

CONVOCATORIA JUNIO (Mayo)

PROFESOR: Eduardo Martínez García

GRUPO 1.2

ÍNDICE

- 1.Explicación de la práctica**
- 2.Lenguaje miniC**
- 3.Análisis léxico**
- 4.Análisis sintáctico**
- 5.Análisis semántico**
- 6.Manual de uso**
- 7.Ejemplo de ejecución**
- 8. Conclusiones e implementaciones hechas**

1. Explicación de la practica

En esta práctica hemos realizado la implementación de un compilador que traduce desde el lenguaje fuente denominado miniC al lenguaje ensamblador de Mips. Este proceso no es nada fácil ya que consta de diferentes fases:

- Análisis de léxico
- Análisis sintáctico
- Análisis semántico
- Generación de código ensamblador

A medida que hemos ido implementado este trabajo hemos ido haciendo uno y cada una de estas fases. El lenguaje que hemos usado para la implementación de este compilador ha sido C en el entorno de Linux, específicamente en VisualStudioCode ya que nos ha ayudado mucho con una extensión llamada "Yash".

Nos hemos apoyado en herramientas que nos han permitido generar este código C a partir de Expresiones Regulares o gramáticas libres de contexto como son Flex (que nos ayudará a generar automáticamente el analizador léxico) y Bison (el cual generará automáticamente el analizador sintáctico).

2 Lenguaje MiniC

MiniC es un lenguaje parecido a C, aunque este es un lenguaje mucho más reducido. Solo maneja constantes y variables enteras. No existen operadores relaciones ni lógicos por lo que solo podemos encontrar las sentencias de control del flujo de ejecución siguientes if, if-else, while y el do-while.

2.1 Símbolos terminales

Dentro de los símbolos terminales de esta gramática podemos encontrar los siguientes: Los enteros los representamos a través de NUME, estos pueden tener un valor desde $[-2^{31}$ hasta $2^{31}]$, también podemos representar las cadenas de texto.

Los identificadores lo representamos a través de id (ID en el código), están formados por secuencias de letras, dígitos, no comenzando por dígito y no excediendo los 16 caracteres.

También podemos encontrar palabras reservadas como print (PRINT en el código), do (DO en el código), var (VAR en el código), if (IF en el código), else (ELSE en el código), read (READ en el código), while (WHILE en el código), const (CONST en el código).

Además de todo esto también tenemos caracteres especiales de separación, operaciones aritméticas, de asignación y control de precedencia y bloques como son: +, -, /; * etc.

2.2 Gramática

Aquí podemos ver la gramática de miniC, que podemos considerarlo una gramática libre de contexto, junto con algunas implementaciones más

program	→	<i>id</i> () { declarations statement_list }
declarations	→	declarations var identifier_list ;
		declarations const identifier_list ;
		λ
identifier_list	→	identifier
		identifier_list , identifier
identifier	→	<i>id</i>
		<i>id</i> = expression
statement_list	→	statement_list statement
		λ
statement	→	<i>id</i> = expression ;
		{ statement_list }
		if (expression) statement else statement
		if (expression) statement
		while (expression) statement
		print (print_list) ;
		read (read_list) ;
print_list	→	print_item
		print_list , print_item
print_item	→	expression
		<i>string</i>
read_list	→	<i>id</i>
		read_list , <i>id</i>
expression	→	expression + expression
		expression - expression
		expression * expression
		expression / expression
		- expression
		(expression)
		<i>id</i>
		<i>num</i>

3. Análisis léxico

Para realizar el análisis léxico, nos hemos apoyado de la herramienta Flex, debido a que así nos lo ha indicado el profesor durante las prácticas.

Mediante este análisis, gracias a los tokens explicados anteriormente, reconocemos las expresiones regulares. En definitiva, sirve para identificar los tokens que representan las expresiones que vamos a utilizar posteriormente. Estos tokens definen tanto las palabras claves para las funciones, los operadores, símbolos, comentarios, etc.

Además, implementamos la detección de errores en modo pánico.

4. Análisis sintáctico

Esta vez, hemos usado la herramienta Bison, herramienta que, como volveremos a comentar posteriormente, es necesaria tanto para el analizador sintáctico como para el semántico.

El analizador sintáctico se encarga de reconocer los ficheros que hemos generado con la gramática anterior. Se estudian además los conflictos producidos por el desplazamiento o reducción.

5. Análisis semántico

Para realizar el analizador semántico usamos los archivos **listaSimbolos.c** y **listaSimbolos.h** y **listaCodigo.c**, **listaCodigo.h**. Hemos definido una tabla de símbolos como una estructura de datos que permita almacenar las variables, constantes y cadenas declaradas en el programa, que nos han proporcionado.

Además, hemos dado de alta variables y constantes para cada una de las declaraciones, debido a que inicialmente solo podíamos usar enteros de tipo "word". El tipo de datos de los valores semánticos que hemos usado ha sido:

```
%union {  
    ListaC código;  
    Char *cadena;  
}
```

Y para incluir la definición de ListaC hemos:

```
%code requires{  
#include "ListaCodigo.h"  
}
```

También hemos controlado que las variables no se declaren más de una vez y no se usen antes de ser declaradas.

6. Generación de código

Para realizar la generación del código, se nos proporcionó la implementación de un tipo lista en c (**listaCodigo** mencionado anteriormente) con la estructura de los datos en la que iremos almacenando el código generado.

Principalmente, hemos añadido en el fichero Bison **sintactico.y** los atributos y acciones semánticas necesarias para generar el código con el formato definido anteriormente en **Gramática**.

Tenemos definidas diferentes funciones para poder representar el código, como **imprimeLs()**, que hace el print del .data, o **imprimirLC(listaC codigo1)**, que imprime la lista del código generado.

Primero, inicializamos los registros al principio del programa y creamos la lista de símbolos con **creaLS()**. Para leer el programa principal imprimiremos la lista de símbolos, con **imprimeLs()**, después concatenamos en la pila las declaraciones y las statement_list, una vez hecho esto imprimimos la lista de código de lo concatenado y pasamos a liberar ambas listas de código y la lista de símbolos.

Es importante conocer el uso de \$\$, que representa el principio de la pila, ya que va a ser usado en la mayoría de las declaraciones. Para cada uno, tenemos un tipo Operación, que se compone de diferentes campos que compondrán una operación (como su propio nombre indica) de nuestro código MIPS. Los campos que hay son: op (código de la operación), res (resultado de la operación), arg1 (primer argumento de la operación) y arg2 (segundo argumento de la operación). Si alguno de ellos no se usa, se marca como NULL. Utilizamos las funciones de las que hemos hablado anteriormente dentro de las declaraciones para conseguir el resultado esperado.

7. Manual de uso

Para compilar el código usamos “make” que atiende a las instrucciones representadas por el archivo makefile.

Una vez disponemos del archivo miniC se hace:

```
./miniC entrada.txt > codigomars.s
```

donde el archivo “entrada.txt” es el fichero de prueba que tenemos para ver la ejecución del código.

Para ver la salida generada en Mars, podemos usar el siguiente comando:

```
./spim -file codigomars.s
```

(Nosotros hemos abierto directamente Mars y hemos ejecutado el código MIPS desde dentro del entorno)

8. Ejemplo de ejecución

En este apartado vamos a poner el ejemplo que hemos utilizado nosotros como entrada.txt y además vamos a poner el código de Mars que crea:

```
prueba() {
const a=0, b=0;
var c=5+2-2;
var dos;
var e = 5;
var i;

print ("Inicio del programa\n");
read(dos);
if (a) print ("a","\n");
else if (b != 0) print ("No a y b\n");
else while (c)
    {
        print ("c = ",c,"\n");
        c = c-2+1;
    }
do{
    print("Bucle do-while", "\n");
    e = e-1;
} while (e != 0);

for (i = 1; i <= 5; i = i + 1) {
    print(i, " ");
}

print ("Final","\n");
}
```

Y el respectivo código.s

```
.data
_a : .word 0
_b : .word 0
_c : .word 0
_dos : .word 0
_e : .word 0
_i : .word 0
$str0 : .asciiz "Inicio del programa\n"
$str1 : .asciiz "a"
$str2 : .asciiz "\n"
$str3 : .asciiz "No a y b\n"
$str4 : .asciiz "c = "
$str5 : .asciiz "\n"
$str6 : .asciiz "Bucle do-while"
$str7 : .asciiz "\n"
$str8 : .asciiz " "
```

```

        $str9 : .asciiz "Final"
        $str10 : .asciiz "\n"
.text
.globl main
main:
    li $t0,0
    sw $t0,_a
    li $t0,0
    sw $t0,_b
    li $t0,5
    li $t1,2
    add $t0,$t0,$t1
    li $t1,2
    sub $t0,$t0,$t1
    sw $t0,_c
    li $t0,5
    sw $t0,_e
    li $v0,4
    la $a0,$str0
    syscall
    li $v0,5
    syscall
    sw $v0,_dos
    lw $t0,_a
    beqz $t0,$l5
    li $v0,4
    la $a0,$str1
    syscall
    li $v0,4
    la $a0,$str2
    syscall
    j $l6
$l5:
    lw $t1,_b
    li $t2,0
    sne $t1,$t1,$t2
    beqz $t1,$l3
    li $v0,4
    la $a0,$str3
    syscall
    j $l4
$l3:
$l1:
    lw $t2,_c
    beqz $t2,$l2
    li $v0,4
    la $a0,$str4
    syscall
    lw $t3,_c
    li $v0,1
    move $a0,$t3
    syscall

```

```

li $v0,4
    la $a0,$str5
    syscall
    lw $t3,_c
    li $t4,2
    sub $t3,$t3,$t4
    li $t4,1
    add $t3,$t3,$t4
    sw $t3,_c
    j $l1
$l2:
$l4:
$l6:
$l7:
    li $v0,4
    la $a0,$str6
    syscall
    li $v0,4
    la $a0,$str7
    syscall
    lw $t0,_e
    li $t1,1
    sub $t0,$t0,$t1
    sw $t0,_e
    lw $t0,_e
    li $t1,0
    sne $t0,$t0,$t1
    beqz $t0,$l8
    j $l7
$l8:
    li $t0,1
    sw $t0,_i
$l9:
    lw $t0,_i
    li $t1,5
    sle $t0,$t0,$t1
    beqz $t0,$l10
    lw $t1,_i
    li $v0,1
    move $a0,$t1
    syscall
    li $v0,4
    la $a0,$str8
    syscall
    lw $t1,_i
    li $t2,1
    add $t1,$t1,$t2
    sw $t1,_i
    j $l9
$l10:
    li $v0,4
    la $a0,$str9

```



```
syscall
li $v0,4
la $a0,$str10
syscall
li $v0 , 10
syscall
```

9.Conclusiones e implementaciones hechas

En un principio pensamos hacer solo una o dos implementaciones, sin embargo, vimos que podíamos hacer la mayor parte de ellas o incluso todas, por lo que hemos decidido hacer todas ellas, en la primera implementación, lo que hemos hecho añadido en statement lo siguiente:

```
| error PC (;) { $$ = creaLC();
                errores ++; }
```

Para ello, hemos creado una nueva lista vacía y añadimos uno a la lista de los errores, además, hemos añadido al código:

```
%define parse.error verbose
```

Para que Bison nos dé más detalles en los errores.

Posteriormente hemos implementado el do-while, para ello hemos creado diferentes etiquetas las cuales representarán cada uno de los saltos que tiene que dar el programa en cada caso que se puede dar en el bucle, en la comprobación de la condición, el salto al inicio de la instrucción, etc. El salto “beqz” se encargará de comprobar el bucle, y “j” es un salto incondicional que nos mandará a retornar al inicio de este.

Otra implementación opcional ha sido añadir las expresiones relacionales. Para hacer estos casos, nos hemos basado en la implementación de las otras operaciones, como la suma, resta, mult, etc. Estas expresiones relacionales han sido útiles de cara a hacer otras funciones como “while” o “if”.

Finalmente, hemos acabado el proyecto con la implementación del for. En la cual lo que hemos hecho ha sido, primero de todo hemos encapsulado el bucle, hemos creado una nueva lista de código para almacenar las operaciones del bucle for, se genera el código para la inicialización del bucle (en este caso 0). Se crea la etiqueta del inicio del bucle, se verifica la condición mediante la expresión relacional, en el caso en el que fuera falsa este saltaría fuera del bucle y en caso de ser verdadera, se ejecuta el código que hemos creado para el interior del bucle, después incrementamos la variable (en este caso es la i) y ya por ultimo volvemos al inicio del bucle con la parte del código con “j”, para finalizar esta implementación lo que hacemos es crear la etiqueta de salto hacia fuera del bucle (la de finalización) y cuando la condición se vuelve falsa el programa continúa la ejecución normal fuera del bucle.

Como conclusiones del proyecto, gracias a esta asignatura, hemos aprendido cosas básicas, pero que no sabíamos hasta ahora, como adentrarnos en como un compilador va haciendo los pasos necesarios para conseguir leer, traducir y ejecutar un código en cualquier lenguaje apropiado.

Nos ha parecido una práctica esencial y muy interesante. Respecto a las clases prácticas, se nos ha hecho fácil no quedarnos sin la posibilidad de avanzar, debido a que, con un poco de constancia, gracias a la implicación del profesor en cada clase, hemos conseguido ir saldando nuestras dudas en clase para llegar al mejor resultado posible.

En definitiva, nos ha parecido una práctica muy acertada, con una carga de tiempo idónea a nuestro parecer, ya que semanalmente tenías que realizar parte del proyecto, pero si poníamos de nuestra parte en cada clase práctica, no teníamos una carga de trabajo excesiva en la recta final del cuatrimestre, como nos ha podido pasar en otras asignaturas.